Wybór architektury i implementacja modelu AI

Szymon Krzysztofik, Tadeusz Kogut

Cel kamienia milowego

Wykorzystanie techniki Transfer Learningu (uczenia transferowego). Załadowanie wstępnie wytrenowanej głębokiej sieci konwolucyjnej (np. ResNet18 lub MobileNetV2) i dostosowanie jej ostatniej warstwy klasyfikującej do liczby ras w naszym zbiorze. Napisanie pętli uczącej i przeprowadzenie procesu trenowania na przygotowanych danych.

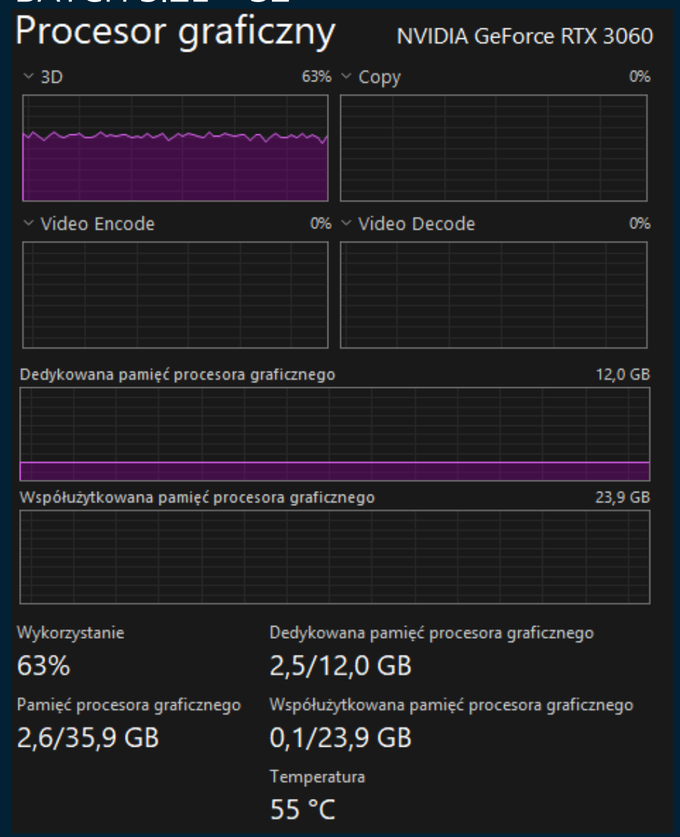
Oczekiwany wynik: Wytrenowany model AI, którego wagi zostały poprawnie zapisane do pliku, gotowy do ewaluacji.

Kod treningu modelu

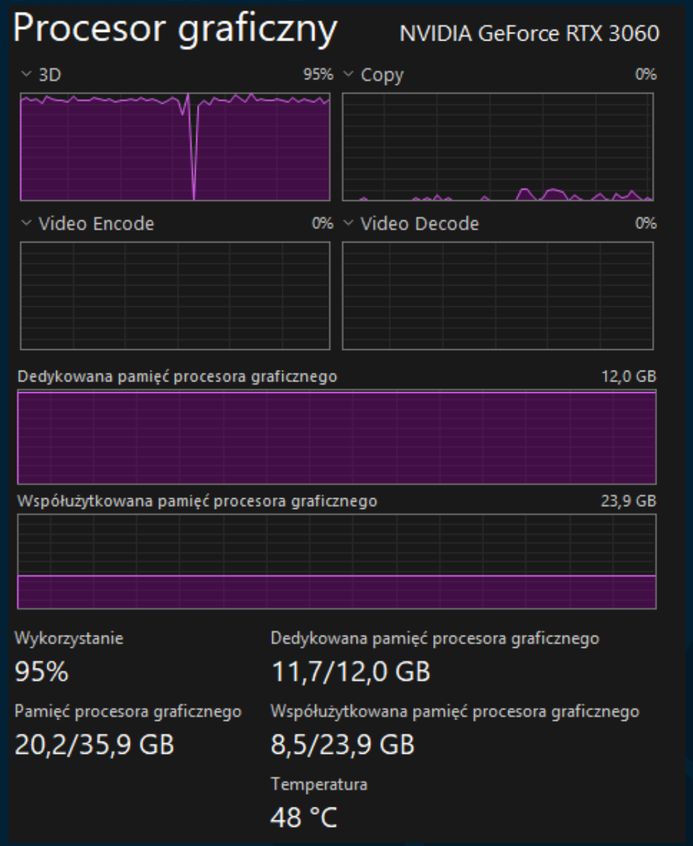
1. **Ustawiamy stałe**

```python
BASE_DIR = Path(__file__).parent.parent
PROCESSED_DIR = BASE_DIR / "data" / "processed"

MODEL_NAME = 'best_model.pth'
SAVE_PATH = BASE_DIR / "saved_models"

PLOT_NAME = 'training_results.png'
PLOT_DIR = BASE_DIR / "reports" / "plots"

BATCH_SIZE = 32          # - rozmiar jednej paczki do nauki
EPOCHS = 20              # - liczba przerobień całego datasetu
LEARNING_RATE = 0.001    # - krok spadku błędu przy optymalizacji
```

BATCH SIZE = 32



BATCH SIZE = 512



Kod treningu modelu

2. **Wybieramy urządzenie do przeprowadzenia obliczeń**

```python
if torch.cuda.is_available():
    device = torch.device("cuda")
else:
    device = torch.device("cpu")
print(f"Do treningu modelu wybrano: {device}")
```

Kod treningu modelu

3. **Przygotowanie danych**

```
train_transforms_data = [
    transforms.Resize((224, 224)),
    transforms.RandomHorizontalFlip(), # - zapobiega przeuczaniu się
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225])
]
train_transforms = transforms.Compose(train_transforms_data)
```

Skalowanie
Losowe Obracanie (przeciw przeuczeniu)
Normalizacja

```
val_transforms_data = [
    transforms.Resize((224, 224)),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225])
]
val_transforms = transforms.Compose(val_transforms_data)
```

Tylko Skalowanie i Normalizacja

Kod treningu modelu

4. **Budowa sztucznej inteligencji**

```
# wybór modelu, nowy head na 120 wyjść (a nie defaultowe 1000), wysłanie do urządzenia
model = models.resnet18(weights=models.ResNet18_Weights.DEFAULT) # (można przetestować resnet50)
num_ftrs = model.fc.in_features
model.fc = nn.Linear(num_ftrs, class_count)
model = model.to(device)

# funkcja błędu i optymalizator
criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.parameters(), lr = LEARNING_RATE)
```

Kod treningu modelu

5. **Główna pętla uczenia maszynowego**

trening

```python
for i in range(EPOCHS):
    model.train() # - tryb nauki
    running_loss = float(0)
    correct_train = int(0)
    total_train = int(0)

    for images, labels in train_loader:
        images = images.to(device)
        labels = labels.to(device)

        optimizer.zero_grad()
        outputs = model(images)
        loss = criterion(outputs, labels)
        loss.backward()
        optimizer.step()
```

1. zerowanie starych błędów
2. predykcja
3. obliczenie błędu
4. obliczenie kierunku poprawy
5. aktualizacja wiedzy sieci

ocena

```python
model.eval() # - tryb oceny
val_loss = float(0)
correct_val = int(0)
total_val = int(0)

with torch.no_grad(): # podczas testów nie aktualizujemy wag
    for images, labels in val_loader:
        images, labels = images.to(device), labels.to(device)
        outputs = model(images)
        loss = criterion(outputs, labels)
```

1. predykcja
2. obliczenie błędu

Wyniki dla modelu resnet18

Specyfikacja Techniczna (Hardware)

Karta graficzna (GPU): Nvidia GeForce RTX 4070 SUPER (12 GB VRAM)
Środowisko: Python 3.12.10, PyTorch 2.11.0 + CUDA 12.1

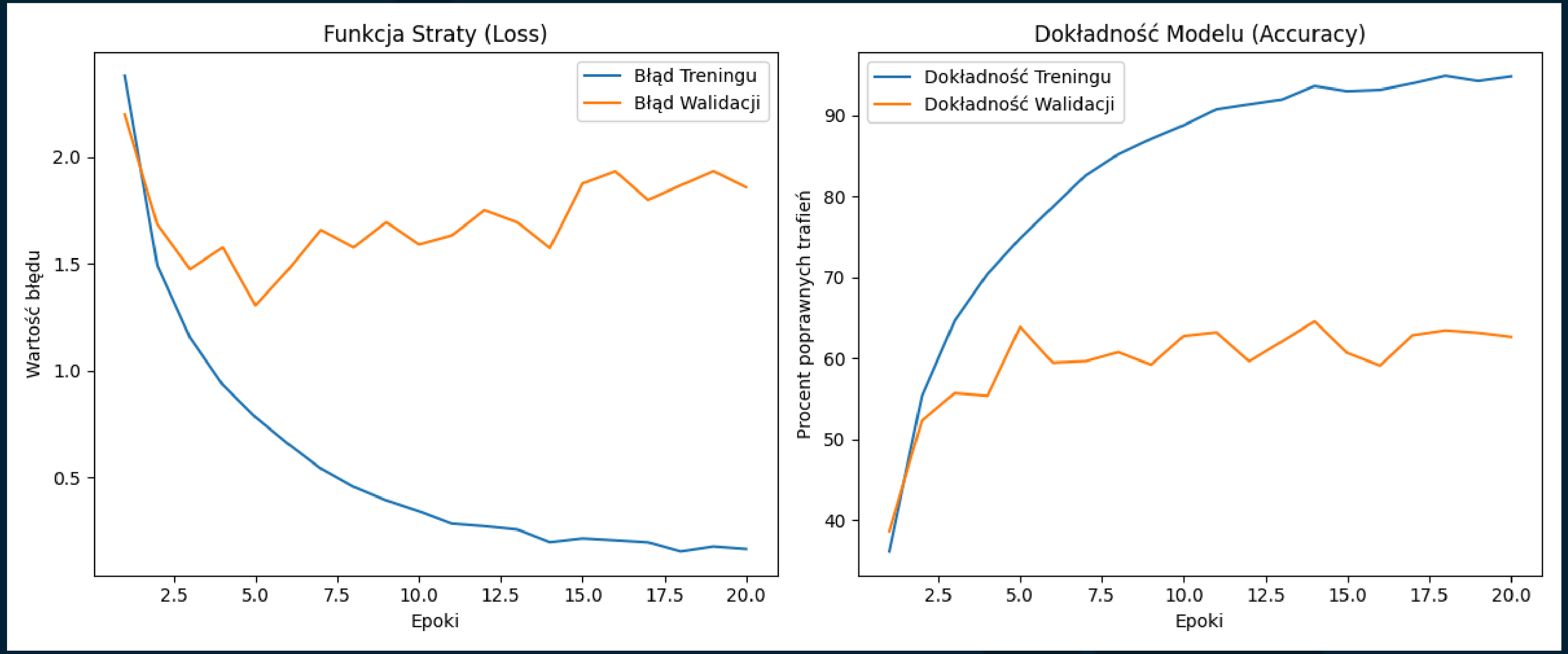
Parametry treningu

Architektura: ResNet18
Liczba epok: 20
Rozmiar partii: 32
Krok spadku błędu przy optymalizacji: 0.001

Uzyskane wyniki

	Trening	Walidacja
Dokładność	94.78%	62.63%
Błąd	0.1645	1.8593

Wykres błędu i dokładności



DZIĘKUJEMY ZA UWAGĘ